

# Conversation Starter for Abacus AI Agent

**Last Updated:** 2024-12-28  
**Purpose:** Critical context for AI agents working on JNX-OS / Qryx project

## Project Overview

**Project Name:** JNX-OS v2 + Qryx (First Product)  
**Type:** Enterprise SaaS Foundation + AI Shopify Sales Assistant  
**Tech Stack:** Next.js 14, Clerk Auth, Supabase (PostgreSQL), Tailwind CSS, Gemini 2.0 Flash  
**Repository:** <https://github.com/JNXLabs/jnx-os>  
**Deployment:** Vercel (<https://www.jnxlabs.ai>)

## Project Location

```
/home/ubuntu/jnx-os/  
├── nextjs_space/           # Next.js application  
│   ├── app/               # Next.js 14 App Router  
│   ├── lib/               # Core libraries  
│   ├── components/        # UI components  
│   ├── public/            # Static assets  
│   └── .env                # ⚠️ PROTECTED - Never commit!  
├── docs/                  # Architecture & guides  
├── scripts/               # Deployment scripts  
└── *.md                   # Documentation files
```

**Working Directory:** `/home/ubuntu/jnx-os/nextjs_space/`

## CRITICAL: Protected Files

**NEVER** modify these files without explicit user approval:

### Core Infrastructure:

- `lib/db/helpers.ts` - Enterprise-grade database operations
- `lib/db/schema-v2.sql` - Database schema
- `app/api/webhooks/clerk/route.ts` - Idempotent webhook handler
- `middleware.ts` - Authentication & RBAC
- `lib/auth/clerk-server.ts` - Server-side auth utilities
- `lib/auth/clerk-client.ts` - Client-side auth utilities

### Security & Privacy:

- `lib/security/rate-limit.ts` - Rate limiting
- `lib/security/headers.ts` - Security headers

- `lib/privacy/*.ts` - GDPR compliance modules
- `lib/observability/logger.ts` - Structured logging

## Qryx Core Files (NEW):

- `lib/db/qryx-helpers.ts` - Qryx database operations
- `lib/ai/gemini.ts` - Gemini AI integration
- `lib/shopify/client.ts` - Shopify API client
- `app/api/qryx/chat/route.ts` - Chat API endpoint
- `app/api/widget/qryx/route.ts` - Widget delivery
- `MIGRATION_QRYX_SHOPIFY.sql` - Qryx database schema

## Configuration:

- `.env` - **HIGHLY SENSITIVE** - Never log or expose
- `.gitignore` - Git exclusion rules
- `next.config.js` - Next.js configuration
- `tailwind.config.ts` - Design system configuration

**Reference:** `docs/BACKEND_CONTRACT.md` for full list and rules



## Environment Variables

**Status Document:** See `ENVIRONMENT_VARIABLES_STATUS.md` for complete overview

## Currently Configured:

### ✓ Clerk Authentication (Production Ready)

```

NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY
CLERK_SECRET_KEY
CLERK_WEBHOOK_SECRET
NEXT_PUBLIC_CLERK_SIGN_IN_URL
NEXT_PUBLIC_CLERK_SIGN_UP_URL
NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL
NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL

```

### ✓ Supabase Database (Production Ready)

```

NEXT_PUBLIC_SUPABASE_URL=https://yxikmojxbiihkpayndw.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY

```

### ✓ Gemini AI (Production Ready)

```

GEMINI_API_KEY=AIzaSyCQBwsACuoGh4X8PUCJ2LmD9-HiCz6qaGU

```

## ⚠️ Shopify API (Needs Vercel Update)

```
SHOPIFY_API_KEY=6e62aef5f8013048ca5b446fa86c6fae
SHOPIFY_API_SECRET=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
SHOPIFY_APP_URL=https://www.jnxfabs.ai # ⚠️ Needs update in Vercel
SHOPIFY_SCOPES=read_products,read_product_listings,read_customers,read_orders
```

### ⚠️ IMPORTANT:

- These variables ARE set locally in `.env`
- Supabase and Clerk ARE deployed to Vercel
- Gemini AI IS deployed to Vercel
- Shopify variables NEED to be updated in Vercel (see `ENVIRONMENT_VARIABLES_STATUS.md` )

## 🗄️ Database Status

### Supabase Project:

- **URL:** `https://yxikmojxbiihkpayndw.supabase.co`
- **Status:** ✔ Active and connected

### Schema Status:

#### ✔ JNX-OS Base Tables (Deployed)

- `orgs` - Organizations (FK: `org_id` )
- `users` - User accounts (FK: `user_id` )
- `audit_logs` - Audit trail
- `billing_customers` - Billing data
- `entitlements` - Feature access
- `feature_flags` - Feature toggles
- `data_export_requests` - GDPR exports

#### ✔ Qryx Tables (Deployed 2024-12-28)

- `shopify_shops` - Shopify store integrations
- `qryx_chat_sessions` - Chat session tracking
- `qryx_chat_messages` - Individual messages
- `qryx_config` - Per-shop configuration
- `conversation_usage` - Usage tracking for billing
- `qryx_product_cache` - Cached Shopify products (no pgvector)
- `qryx_analytics_daily` - Pre-aggregated analytics

### Recent Migrations:

- ✔ **2024-12-28:** `MIGRATION_QRYX_SHOPIFY.sql` executed successfully
- ✔ **Fixed:** Foreign key references (`org_id`, `user_id`)
- ✔ **Removed:** pgvector dependency (optional feature)

### Verification:

```
cd /home/ubuntu/jnx-os/nextjs_space
node test-supabase.js # Tests base connection
node test-qryx-shop.js # Tests Qryx tables
```

## Current Project Phase

### Phase 4: Qryx Chat Widget & API Endpoints COMPLETE

**Status:** Backend and database infrastructure complete, awaiting Shopify configuration

#### Completed Components:

1. **Database Schema** 
  - 7 Qryx-specific tables
  - Foreign key relationships to JNX-OS
  - GDPR-compliant soft deletes
  - Performance indexes
2. **AI Integration** 
  - Gemini 2.0 Flash configured
  - Cost tracking implemented
  - Product context building
  - `lib/ai/gemini.ts`
3. **Shopify Integration** 
  - OAuth flow implementation
  - Product fetching
  - Script tag injection
  - `lib/shopify/client.ts`
4. **API Endpoints** 
  - `/api/qryx/chat` - AI chat handler
  - `/api/qryx/config` - Configuration management
  - `/api/widget/qryx` - Widget JavaScript delivery
  - `/api/shopify/install` - OAuth initiation
  - `/api/shopify/callback` - OAuth callback
5. **Dashboard UI** 
  - `/app/qryx` - Main dashboard
  - Overview tab with widget code
  - Configuration tab (appearance + AI settings)
  - Analytics tab (placeholder)
6. **Database Helpers** 
  - `lib/db/qryx-helpers.ts`
  - Shop management functions
  - Chat session operations
  - Configuration CRUD

## Pending Actions:

1.  **Update Shopify variables in Vercel**
    - Set `SHOPIFY_APP_URL=https://www.jnslabs.ai`
    - Deploy other Shopify variables
    - See `ENVIRONMENT_VARIABLES_STATUS.md` for details
  2.  **Configure Shopify Partner Dashboard**
    - Set App URL
    - Configure Redirect URIs
    - Verify API scopes
  3.  **Redeploy to Vercel**
    - After environment variable updates
    - Without build cache
  4.  **Test Installation Flow**
    - Click "Install Qryx on Shopify"
    - Complete OAuth flow
    - Verify dashboard loads
- 



## Current App Status

---

### Working Features:

#### **Authentication & Authorization**

- Clerk integration (signup/login)
- Enterprise-grade idempotent webhooks
- Role-based access control (admin/member)
- Multi-tenant architecture via Clerk Organizations

#### **Core Application**

- Landing page ( / )
- Dashboard ( /app )
- Admin dashboard ( /admin ) - admin role required
- User settings ( /app/settings )
- Products page ( /products )
- Privacy & Terms pages

#### **Qryx Infrastructure**

- Database schema deployed
- API endpoints implemented
- Dashboard UI built
- AI integration configured
- Shopify client ready



### In Progress

- Shopify OAuth installation flow (pending env var updates)
- Widget deployment to Shopify stores
- Live chat testing

## Known Issues:

**None currently** - All previous issues resolved:

- ☒ Fixed "Configuration Error" (database tables missing)
- ☒ Fixed foreign key errors (org\_id, user\_id)
- ☒ Fixed pgvector dependency error
- ☒ Removed `.env` from Git history



## Design System: JNX Dark

### Color Palette:

```
/* Primary */
--jnx-primary: #06b6d4; /* Cyan-500 */
--jnx-secondary: #0891b2; /* Cyan-600 */

/* Backgrounds */
--jnx-dark: #0f172a; /* Slate-900 */
--jnx-darker: #020617; /* Slate-950 */
--jnx-card: #1e293b; /* Slate-800 */

/* Accents */
--jnx-accent: #3b82f6; /* Blue-500 */
```

### Key Components:

- `ButtonPrimary` - Cyan gradient, optional glow
- `ButtonSecondary` - Subtle slate styling
- `InputField` - Dark theme, cyan focus ring
- `FeatureCard` - Glassmorphism effect
- `TerminalBox` - Monospace code display
- `StatusBadge` - Color-coded status indicators

**Reference:** `tailwind.config.ts`, `app/globals.css`



## Development Workflow

### Local Development:

```
# Start dev server
cd /home/ubuntu/jnx-os/nextjs_space
yarn dev

# Open in browser (automatic via DeepAgent)
http://localhost:3000

# Run tests (when available)
yarn test

# Type checking
yarn tsc --noEmit
```

## Database Operations:

```
# Test connection
node test-supabase.js

# Verify Qryx tables
node test-qryx-shop.js

# Run migrations (in Supabase SQL Editor)
# Execute SQL from: MIGRATION_QRYX_SHOPIFY.sql
```

## Git Workflow:

```
# Check status
git status

# Stage changes (NEVER stage .env)
git add <files>

# Commit with descriptive message
git commit -m "feat: Add feature description"

# Push to main
git push origin main

# Vercel auto-deploys from main branch
```

## Deployment:

1. Push to GitHub main branch
2. Vercel auto-deploys
3. Check deployment logs in Vercel Dashboard
4. Verify at <https://www.jnxxlabs.ai>



## Essential Documentation

### Getting Started:

- `README.md` - Project overview
- `QUICKSTART.md` - 5-minute setup guide
- `SETUP.md` - Detailed setup instructions
- `ENVIRONMENT_VARIABLES_STATUS.md` - **NEW** - Complete env var status

### Architecture & Design:

- `docs/ARCHITECTURE.md` - System architecture
- `docs/BACKEND_CONTRACT.md` - Development rules
- `docs/QRYX_SHOPIFY_ARCHITECTURE.md` - Qryx-specific architecture

### Database:

- `lib/db/schema-v2.sql` - JNX-OS base schema
- `MIGRATION_QRYX_SHOPIFY.sql` - Qryx schema

- `MIGRATION_SIMPLE.sql` - Base migrations
- `CRITICAL_SCHEMA_RESTORE.md` - Schema verification

## Product Documentation:

- `docs/QRYX_PRICING_STRATEGY.md` - Qryx pricing model
- `JNX_LEARNING_PLATFORM_DEVELOPER_QUICKSTART.md` - Product SDK guide

## Compliance:

- `docs/GDPR_COMPLIANCE.md` - GDPR implementation
- `app/privacy/page.tsx` - Privacy policy
- `app/terms/page.tsx` - Terms of service

## Project History:

- `DELIVERY_SUMMARY.md` - Phase completion summary
- `ANALYSIS_REPORT.md` - Problem analysis & solutions



## AI Agent Guidelines

### DO:

- ☒ Always check `ENVIRONMENT_VARIABLES_STATUS.md` before asking about env vars
- ☒ Read `BACKEND_CONTRACT.md` before modifying core files
- ☒ Use `yarn` as package manager (NOT npm)
- ☒ Test locally before committing
- ☒ Follow JNX Dark design system
- ☒ Maintain GDPR compliance
- ☒ Use structured logging
- ☒ Implement idempotent operations
- ☒ Check file summaries in system context

### DON'T:

- ☒ Modify protected files without approval
- ☒ Commit `.env` or secrets to Git
- ☒ Log sensitive data (API keys, user PII)
- ☒ Skip database migrations
- ☒ Use npm/npx (use yarn)
- ☒ Break existing authentication flows
- ☒ Remove error handling or logging
- ☒ Assume env vars - verify first

### Before Making Changes:

1. Read relevant documentation
2. Check if file is protected
3. Understand current implementation
4. Test locally
5. Verify no breaking changes

6. Update documentation if needed

## After Making Changes:

1. Test functionality locally
2. Run type checking
3. Commit with clear message
4. Push to GitHub
5. Monitor Vercel deployment
6. Verify in production
7. Update documentation

## Quick Command Reference

### Package Management:

```
yarn install      # Install dependencies
yarn add <package> # Add package
yarn remove <package> # Remove package
```

### Development:

```
yarn dev      # Start dev server
yarn build    # Production build
yarn start    # Start production server
yarn lint     # Run ESLint
```

### Database Testing:

```
node test-supabase.js      # Test DB connection
node test-qryx-shop.js     # Test Qryx tables
node verify-schema-simple.js # Verify base schema
node verify-migration.js    # Verify Qryx migration
```

### Git:

```
git status      # Check status
git add .       # Stage all (careful with .env!)
git commit -m "... " # Commit
git push origin main # Push to GitHub
```

## Next Steps for Agent

### Immediate Actions:

1. Read this document completely
2. Review `ENVIRONMENT_VARIABLES_STATUS.md`

3. Check `BACKEND_CONTRACT.md`
4. Understand current phase status

### When User Continues Development:

1. Ask if Shopify env vars are updated in Vercel
2. Ask if Shopify Partner Dashboard is configured
3. Proceed with testing installation flow
4. Help debug any issues that arise

### Common User Requests:

- “Update Shopify configuration” → Guide through env var setup
- “Test Qryx installation” → Verify env vars first, then test flow
- “Widget not appearing” → Check OAuth flow, Script Tag installation
- “Chat not working” → Verify Gemini API, database connections
- “Dashboard errors” → Check Supabase tables, authentication

## Success Criteria

### Phase 4 Complete When:

-  Database schema deployed
-  API endpoints functional
-  Dashboard UI complete
-  Shopify OAuth flow tested
-  Widget installed on test store
-  Chat functionality verified
-  Configuration changes reflected in widget

### Production Ready When:

-  All environment variables configured
-  End-to-end testing complete
-  Error handling verified
-  Performance optimized
-  Documentation updated
-  Security audit passed

## CRITICAL UPDATE (2024-12-29): Installation Flow Redesign

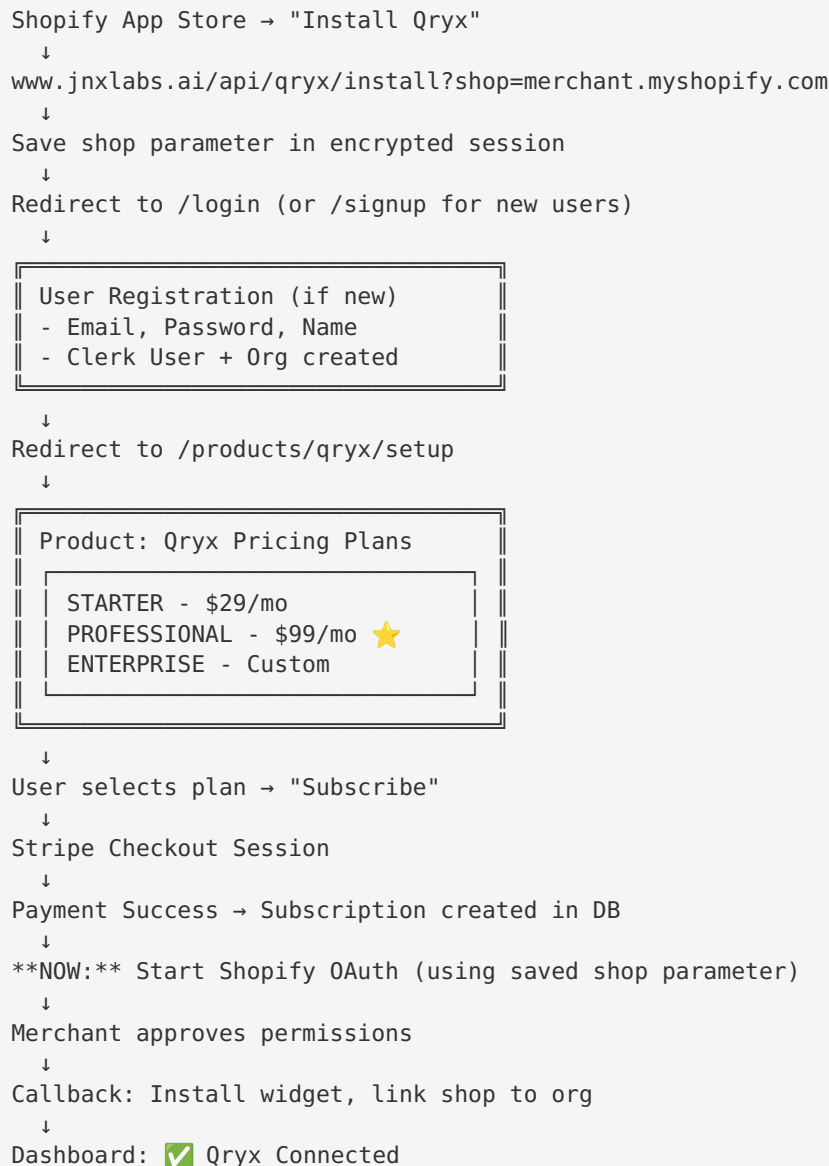
### What Was Wrong:

The original implementation assumed a **direct Shopify OAuth installation**:

Shopify App Store → Install → OAuth → Done

**This is fundamentally incorrect** for a SaaS business model.

## ✓ Correct Multi-Product SaaS Flow:



## 🔧 Required Components (Not Yet Built):

### 1. Product Selection Page ✗

- File: `/app/products/qryx/setup/page.tsx`
- Shows Qryx pricing tiers
- "Subscribe" button per plan
- Checks if user already has subscription

### 2. Stripe Integration ✗

- Files needed:
- `lib/stripe/client.ts` - Stripe SDK setup
- `app/api/stripe/checkout/route.ts` - Create checkout session
- `app/api/stripe/webhook/route.ts` - Handle subscription events
- `lib/db/billing-helpers.ts` - Subscription CRUD
- Webhooks to handle:

- `checkout.session.completed`
- `customer.subscription.updated`
- `customer.subscription.deleted`
- `invoice.payment_succeeded/failed`

### 3. Shop Session Management ❌

- File: `lib/session/shop-session.ts`
- Purpose: Preserve `shop` parameter through Login → Products → Payment → OAuth
- Implementation: Encrypted cookie or DB state

### 4. Installation Flow Update ⚠️ MUST CHANGE

- File: `app/api/qryx/install/route.ts`
- Current (WRONG): Direct redirect to Shopify OAuth
- Required: Save shop → Redirect to login → ... → OAuth after payment

### 5. OAuth Trigger After Payment ❌

- New file: `app/api/qryx/start-oauth/route.ts`
- Called after Stripe payment success
- Validates subscription → Retrieves shop → Initiates OAuth

### 6. Subscription Management UI ❌

- File: `/app/app/billing/page.tsx`
- Features:
  - Current plan display
  - Usage stats (conversations used / limit)
  - Upgrade/Downgrade options
  - Cancellation flow



## Implementation Priority:

#### Phase 5A - Core Flow (NEXT):

1. Shop session management
2. Installation flow redirect update
3. Product selection page (basic)
4. Stripe checkout integration
5. OAuth trigger after payment

#### Phase 5B - Polish:

1. Subscription management UI
2. Billing dashboard
3. Usage tracking
4. Plan upgrade/downgrade



## Test Shop:

- Domain: `shopbotv3.myshopify.com`
- Use for end-to-end testing



## Current Blockers:

- ❌ No Stripe integration
- ❌ No product selection page
- ❌ Installation flow goes directly to OAuth (wrong)
- ❌ No billing/subscription management

## Success Criteria for Phase 5:

- User can install Qryx from Shopify App Store
- User is prompted to login/register
- User can select and purchase a pricing plan
- Payment is processed via Stripe
- OAuth happens AFTER successful payment
- Widget is installed automatically
- Dashboard shows subscription status

---

**This document is your starting point. Read it carefully before beginning work on the project.**

**Last Updated:** 2024-12-29 12:15 UTC

**Maintained By:** Abacus AI DeepAgent

**Next Review:** After Phase 5A implementation